CodeClash: A Real-Time Multiplayer Coding and Quiz Contest Platform

CS455 Project: Requirements and Architecture Document

Ridham Divyakant Solanki	*230853*
Peeyush Agarwal	*230749*
Srujan Sachin Bhirud	*231039*
Aritra Kolay	*230192*
Om Chandrakant Chaudhari	*230715*

September 26, 2026

Contents

1 Requirements Specification **4**

1.1 Use-Case Diagram 4

1.2 User Stories 5

1.3 Functional Requirements 9

1.4 Non-Functional Requirements 11

1.5 Initial Requirements Traceability Matrix 13

2 Architecture and Design **15**

2.1 Component Diagram 15

2.2 Sequence Diagrams 17

2.3 Design-Pattern Mapping 21

2.4 Architecture Alternatives and Rejected Decisions 21

3 Project Management Setup **24**

3.1 Jira Project and Initial Backlog 24

3.2 Jira–GitHub Integration 26

3.3 Initial Sprint Plan 28

4 AI Engineering Log for Requirements and Architecture Activities **31**

4.1 AI-Assisted Requirements Refinement and Planning 32

4.1.1 Objective 32

4.1.2 Prompt / Interaction 32

4.1.3 LLM Output 32

4.1.4 Team Evaluation 32

4.1.5 Changes Made . . . 33
4.1.6 Final Decision . . . 33
4.2 AI-Assisted Architecture Generation . . . 33
4.2.1 Objective . . . 33
4.2.2 Prompt / Interaction . . . 33
4.2.3 LLM Output . . . 33
4.2.4 Team Evaluation . . . 33
4.2.5 Changes Made . . . 34
4.2.6 Final Architecture . . . 34
4.3 AI-Assisted Project Setup . . . 35
4.3.1 Objective . . . 35
4.3.2 Prompt / Interaction . . . 35
4.3.3 LLM Output . . . 35
4.3.4 Team Evaluation and Final Decision . . . 35
4.4 Lessons . . . 35

1 Requirements Specification

CodeClash runs live coding and quiz contests for hundreds of participants at once. An organiser opens registration with a fixed number of seats. At start time, coders submit solutions that are compiled and judged in isolation against hidden tests, and quiz players answer server-timed rounds scored on speed. Every verdict re-ranks the room, and the new standings reach every connected browser within a second. Between contests the problem library stays open as a practice archive.

Three resources are contested and must never be double-allocated: contest *seats* (a fixed capacity), *judge slots* (execution units that run one submission at a time), and the *first-solve bonus* for each problem. The requirements below are written so that each of these rules can be tested directly.

1.1 Use-Case Diagram

Figure 1 shows the use cases of CodeClash and the actors who interact with them. The Participant, Problem setter, Organiser and Platform admin are primary human actors, each with a separate area of the web client (`/arena`, `/authoring`, `/control` and `/admin`). The *judge worker* is a secondary system actor that runs submissions in isolation and returns verdicts. The use cases in the lower band of the diagram involve the judge worker. Use cases UC-10 and UC-15 to UC-20 are optional and will be built only if the mandatory baseline is finished early.

Table 1 lists every use case with its actors, the requirements it realises and whether it is in the mandatory scope for the 14 November finish.

Table 1: Use-case catalogue

ID	Use case	Primary actor	Secondary actor	Requirements	Scope
UC-01	Sign up / sign in	All roles	–	FR-01, FR-02	Mandatory
UC-02	Practise from archive	Participant	Judge worker	FR-19	Mandatory
UC-03	Reserve seat	Participant	–	FR-06, FR-11	Mandatory
UC-04	Join waitlist («extend» UC-03)	Participant	–	FR-07	Mandatory
UC-05	Modify / withdraw seat	Participant	–	FR-08, FR-09	Mandatory
UC-06	Enter live contest (a reserved seat is enough; check-in is optional)	Participant	–	FR-10, FR-15	Mandatory
UC-07	Submit code solution	Participant	Judge worker	FR-15 – FR-17	Mandatory
UC-08	Answer quiz round	Participant	–	FR-20	Mandatory
UC-09	View live leaderboard	Participant	–	FR-21, FR-22	Mandatory
UC-10	Ask clarification	Participant	–	FR-26	Optional
UC-11	Author and version problem	Problem setter	–	FR-12	Mandatory
UC-12	Upload tests and reference	Problem setter	–	FR-13	Mandatory

ID	Use case	Primary actor	Secondary actor	Requirements	Scope
UC-13	Run publish check	Problem setter	Judge worker	FR-14	Mandatory
UC-14	Create and configure contest	Organiser	–	FR-04, FR-05	Mandatory
UC-15	Answer clarifications	Organiser	–	FR-26	Optional
UC-16	Rejudge / invalidate	Organiser	Judge worker	FR-18	Optional
UC-17	Publish standings	Organiser	–	FR-23 – FR-25	Optional
UC-18	Manage roles	Platform admin	–	FR-03	Optional
UC-19	Manage judge workers	Platform admin	–	FR-27	Optional
UC-20	Search audit trail	Platform admin	–	FR-28	Optional

1.2 User Stories

The stories below describe the mandatory behaviour of CodeClash. Each story names its user-story ID (used in the traceability matrix and in Jira), its requirements and its estimate in story points (Fibonacci scale). The acceptance criteria are written into each story so that every story can be checked by a test.

Story 1: Account Creation and Role-Based Access (US-01, US-02; FR-01, FR-02; 3+3 pts)

Neha signs up for CodeClash with her email and a password, and lands on the `/arena` page as a participant. Her session uses a short-lived access token and a rotating refresh token. A wrong password shows an error and repeated attempts are rate-limited. Out of curiosity, she calls an `/admin` API directly; the server answers `403 Forbidden`, because every role check is done on the server and hiding menu items is never treated as a security control.

Story 2: Creating and Scheduling a Contest (US-04, US-05; FR-04, FR-05; 5+3 pts)

Karan, an organiser, creates a coding contest with 200 seats, a start and end time, the ICPC scoring mode and a freeze time. The contest is saved as *Draft*; invalid values such as a capacity of zero or an end time before the start are rejected. At the configured server times the scheduler moves the contest to *Registration open*, then *Running*, *Frozen* and *Ended*. If Karan tries to jump from Draft straight to Running, the server returns `409 Conflict`.

Story 3: Reserving a Seat Under Concurrency (US-06; FR-06, FR-11, NFR-04; 8 pts)

Only one seat is left when Aman and Sara click *Reserve* at the same moment. Exactly one of them gets *Reserved* and the other goes to the waitlist; the contest never holds more reserved seats than its capacity. Aman's browser then retries his request after a network glitch; the

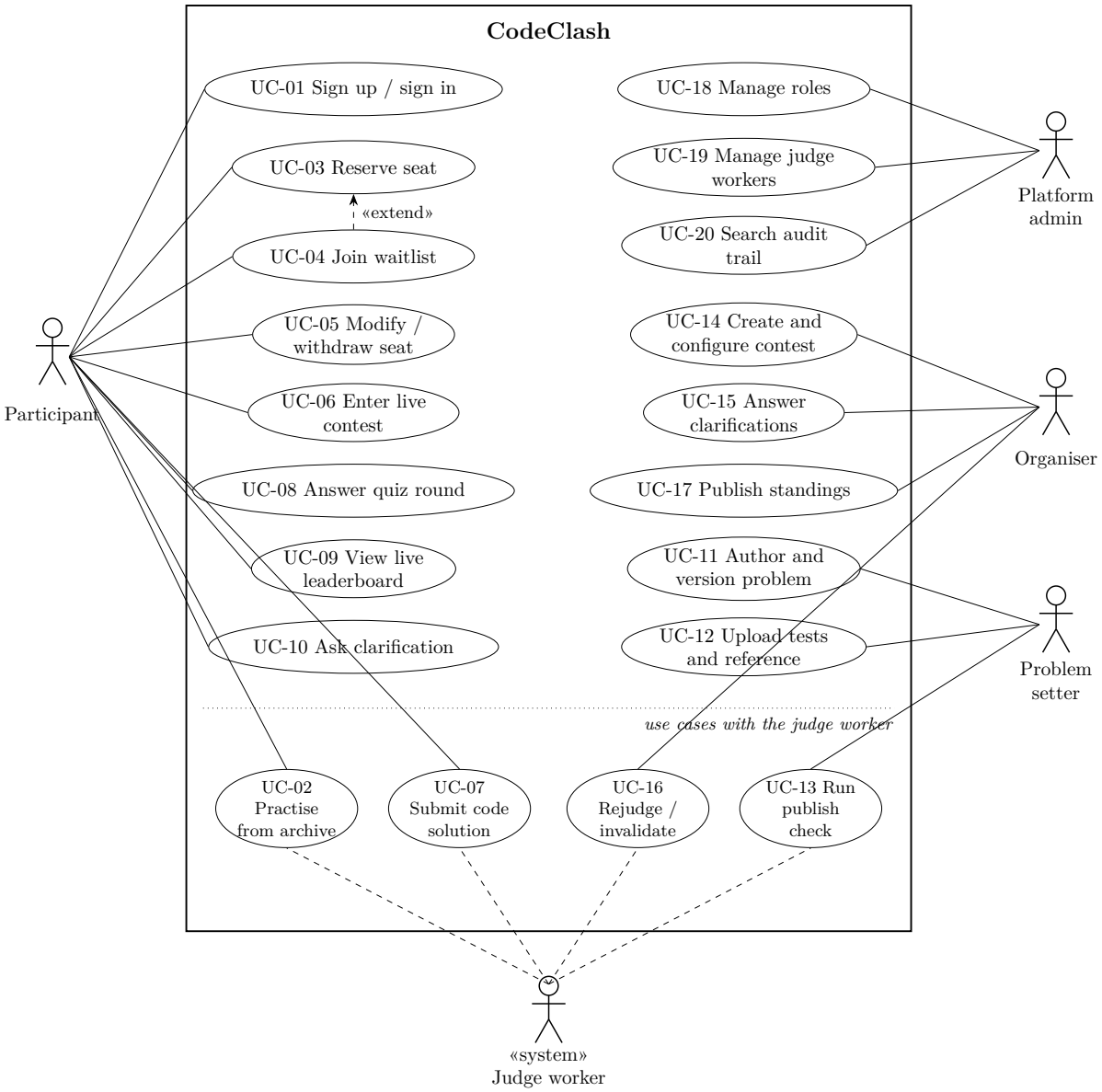


Figure 1: Use-case diagram of CodeClash. Solid lines join human actors; dashed lines join the judge worker.

second request returns the seat he already holds instead of creating a new one. The database decides who gets the last seat, so two requests can never both take it.

Story 4: Joining the Waitlist (US-07; FR-07; 3 pts)

Sara clicked *Reserve* after the last seat was taken. Instead of an error she is placed on the waitlist and shown her position. Positions are strictly first-in first-out.

Story 5: Withdrawing and Automatic Promotion (US-08; FR-09; 5 pts)

Aman can no longer attend and withdraws. Because the waitlist is not empty, the person at its head (Sara) is promoted to *Reserved* in the same transaction and is notified live; the reserved count does not change, since one seat was released and one was taken. Later, when the waitlist is empty, another participant withdraws and the reserved count goes down by one. In both cases the reserved count equals the number of reserved seats after the commit.

Story 6: Authoring and Versioning a Problem (US-10, US-11; FR-12, FR-13; 5+5 pts)

Divya, a problem setter, writes a problem statement with samples and tags. Every save creates a new immutable version, and a contest pins the version it uses, so later edits never break a running contest. She uploads a zip of `.in`/`.out` test pairs, a reference solution and per-language time and memory limits. The reference solution must pass all tests before the upload is accepted.

Story 7: Publish Check Decided by the Sandbox (US-12; FR-14; 5 pts)

Divya asks to publish the problem. Along with her reference solution she has uploaded two wrong solutions: a slow quadratic one and one that ignores an empty input. The system runs all three in the sandbox against every test. The reference passes, but the quadratic solution also passes every test, so the publish check fails and the report says which wrong solution survived. Divya adds a maximum-size test, the quadratic solution now gets TLE, and the check passes. The *Publish* button is enabled only on a pass, and the decision is taken only from the sandbox verdicts.

Story 8: Submitting Code and Receiving a Verdict (US-13; FR-15, FR-16; 8 pts)

During the contest, Neha submits a Python solution from the Monaco editor. She never checked in; her reserved seat is enough to enter the contest and submit. The server replies at once with `202 Accepted` and a submission ID. A judge worker compiles the code once and runs it in isolation against the hidden tests, stopping at the first failure. The verdict (AC, WA, TLE, MLE, RE or CE) appears live in her browser; for a compile error she also sees the compiler output.

Story 9: Judging Exactly Once When a Worker Dies (US-14; FR-17, NFR-05; 8 pts)

A judge worker crashes halfway through running Neha's submission. After the visibility timeout another worker reclaims the queue entry with a new claim token and judges it. When the first worker comes back and tries to save its old result, the database update matches no document,

so nothing changes. The nightly invariant check finds exactly one committed verdict for every submission.

Story 10: Server-Timed Quiz Rounds (US-17; FR-20, NFR-16; 5 pts)

In a quiz round, each question is open for 30 seconds by the server clock. Priya answers correctly after 6 seconds and scores more than Arjun, who answers correctly after 25 seconds. An answer that reaches the server after the window closes is rejected, whatever the player's own clock shows.

Story 11: Live Leaderboard and First-Solve Bonus (US-18; FR-21, FR-22, NFR-01; 8 pts)

Every committed verdict updates the standings, and the change reaches all 200 connected browsers within one second at the 95th percentile. Two participants get an accepted verdict on the same problem at almost the same moment; the first-solve bonus is awarded exactly once, to the earlier submission.

Story 12: Practising Without Hurting a Live Contest (US-24; FR-19; 3 pts)

While a contest is running, Arjun practises on archived problems. His practice verdicts never change any standings or ratings, and practice submissions use at most a fifth of the judge slots, so contest submissions are never starved. When he tries to open a problem that belongs to a contest that has not been published yet, the archive returns `403 Forbidden`.

Optional stories. The following stories are in the Jira backlog with the label `optional` and will be built only if the four sprints finish early.

Table 2: Optional user stories

ID	User story	Req.	Pts
US-03	As an admin, I want to grant and revoke roles, with every change audited.	FR-03, FR-28	2
US-09	As a participant, I want to edit my registration details without losing my seat.	FR-08	2
US-16	As an organiser, I want to rejudge or invalidate submissions after fixing a test.	FR-18	5
US-19	As an organiser, I want to freeze and later publish standings.	FR-23, FR-24	3
US-20	As a participant, I want to ask clarifications, and as an organiser I want to answer them privately or for everyone.	FR-26	5
US-22	As an admin, I want to register, drain and evict judge workers.	FR-27, NFR-14	5
US-23	As an admin, I want to search the audit trail by actor, action, contest and time.	FR-28, NFR-10	3
US-25	As a participant, I want my rating to update after each published contest.	FR-25	3
US-26	As a participant, I want to check in to confirm I will attend; check-in is never needed to submit.	FR-10	3

1.3 Functional Requirements

Each requirement is tagged with its module and priority. *Must* requirements form the mandatory baseline for 14 November; *Optional* requirements are frozen until that baseline is complete.

FR-01 *(Identity, Must)* Users shall be able to create an account and sign in; sessions use short-lived JWT access tokens and rotating refresh tokens.

FR-02 *(Identity, Must)* The server shall enforce four roles (participant, problem setter, organiser, platform admin) on every endpoint and on the `/arena`, `/authoring`, `/control` and `/admin` areas.

FR-03 *(Identity, Optional)* A platform admin shall be able to grant and revoke roles; every change is written to the audit trail.

FR-04 *(Contest, Must)* An organiser shall be able to create a contest with a type (coding, quiz or mixed), seat capacity, schedule, scoring mode and freeze time.

FR-05 *(Contest, Must)* A contest shall move through Draft → Registration open → Running → Frozen → Ended → Published; illegal transitions shall be rejected.

FR-06 *(Registration, Must)* A participant shall be able to reserve a seat; the system shall never reserve more seats than the contest capacity.

FR-07 *(Registration, Must)* When a contest is full, the participant shall be placed on a first-in first-out waitlist and shown their position.

FR-08 *(Registration, Optional)* A participant shall be able to modify their registration (display name, preferred language) without losing their seat.

FR-09 *(Registration, Must)* Withdrawing a reserved seat shall run in one transaction: if the waitlist has a head, that user is promoted and the reserved count is unchanged; if the waitlist is empty, the reserved count is decremented. A promoted user is notified live.

FR-10 *(Registration, Optional)* A participant may check in during the check-in window to confirm attendance. Check-in is never required: a reserved seat alone allows entering the contest and submitting, and seats are never released for missing check-in.

FR-11 *(Registration, Must)* A user shall hold at most one active seat per contest.

FR-12 *(Problem, Must)* A problem setter shall be able to author problems (statement, samples, tags); every save creates a new immutable version.

FR-13 *(Problem, Must)* A setter shall be able to upload hidden tests, a reference solution and optional wrong solutions, and set time and memory limits per language. The reference must pass all tests before the upload is accepted.

FR-14 *(Problem, Must)* A problem version shall be published only after a publish check passes: in the sandbox, the reference solution gets AC on every test, and every setter-supplied wrong solution fails at least one test. The pass or fail decision is taken only from sandbox verdicts.

FR-15 *(Submission, Must)* While a contest is running, any participant with a reserved seat shall be able to submit code in the supported languages (JavaScript and Python 3, which the existing judge already runs; C++ is optional) from the Monaco editor, whether or not they checked in.

FR-16 *(Submission, Must)* Each submission shall be compiled and run in isolation against hidden tests and receive one verdict: AC, WA, TLE, MLE, RE or CE.

FR-17 *(Submission, Must)* Each submission shall be judged exactly once, even if the queue redelivers it or its worker dies mid-run.

FR-18 *(Submission, Optional)* An organiser shall be able to rejudge or invalidate one submission or all submissions for a problem.

FR-19 *(Submission, Must)* Users shall be able to practise on archived problems through the same judge. Practice never affects standings or rating; while a contest is running, contest submissions keep a reserved share of judge slots (at least 80%, so practice uses at most 20%); a problem attached to an unpublished contest cannot be opened or submitted from the archive.

FR-20 *(Quiz, Must)* Quiz rounds shall be timed by the server; answers outside the server window are rejected; a correct answer scores by the quiz speed rule below.

FR-21 *(Scoring, Must)* Every committed verdict shall update the standings using the ICPC rule below (unsolved problems add no penalty); the first-solve bonus for a problem is awarded exactly once.

FR-22 *(Leaderboard, Must)* Standings changes shall be pushed to all connected browsers in real time.

FR-23 *(Leaderboard, Optional)* After freeze time, participants shall see frozen standings while organisers still see live standings.

FR-24 *(Leaderboard, Optional)* An organiser shall publish final standings, which unfreezes the board and locks results.

FR-25 *(Rating, Optional)* After a contest is published, each participant's rating shall be updated and shown on their profile.

FR-26 *(Clarification, Optional)* A participant shall be able to ask a clarification; organisers can answer privately or broadcast to everyone.

FR-27 *(Admin, Optional)* A platform admin shall be able to register, drain and evict judge workers and see live slot usage.

FR-28 *(Audit, Must)* Every privileged action (role change, contest state change, rejudge, publish) shall be written to an append-only audit collection (a search screen is optional).

Scoring rules referenced by FR-20 and FR-21. *ICPC rule.* For a participant p, $\mathrm{solved}(p)$ is the number of problems on which p has an accepted verdict before the contest ends, and

$$\mathrm{penalty}(p) = \sum_{i \,\in\, \mathrm{solved}(p)} \left(t_i + 20\, w_i\right),$$

where t_i is the number of whole minutes from the contest start to p's first AC on problem i, and w_i is the number of rejected submissions on i before that AC, not counting compile errors. An unsolved problem adds zero penalty, however many wrong attempts it has. Ranking is by solved (descending), then penalty (ascending), then the earlier time of the last AC.

Quiz speed. For question q with base points B_q (default 1000) and answer window T_q, a correct answer received t seconds after the question opened (both times measured on the server) scores

$$\mathrm{score}(q) = \mathrm{round}\left(B_q \left(1 - 0.5\,\frac{t}{T_q}\right)\right), \qquad 0 \le t \le T_q,$$

so the score falls from B_q (instant) to $B_q/2$ (at the deadline). A wrong answer scores 0 and an answer with $t > T_q$ is rejected.

First-solve bonus. For each problem, the winner is the AC with the earliest submission time (ties broken by the lower submission ID). In quiz and IOI modes the winner receives 10% of the problem's maximum points; in ICPC mode the bonus is a badge only and never changes solved or penalty. If a rejudge removes the winning AC, the bonus moves to the next earliest AC.

1.4 Non-Functional Requirements

Every non-functional requirement has a number and a method of verification, so that it can be passed or failed.

NFR-01 Leaderboard latency. A standings change shall reach connected browsers within 1 s at p95 after the verdict commits, with 200 clients connected. Verified by a load test with a Playwright timing probe.

NFR-02 Judge turnaround. Judge turnaround shall be at most 10 s at p95 under normal contest load. Verified by queue metrics.

NFR-03 API responsiveness. Non-judge API endpoints shall respond within 300 ms at p95 at 200 requests per second. Verified by a load test.

NFR-04 Seat correctness. When more reservation requests arrive at the same time than seats remain, the reserved count never exceeds capacity and no user holds two active seats. Verified by a concurrent integration test and an invariant query.

NFR-05 Judging correctness. Every submission shall have exactly one committed verdict, even when a worker is killed mid-run, with no orphaned claims. Verified by an integration test that kills a worker and an invariant query.

NFR-06 Execution isolation. Submitted code shall run with no network access, CPU-time and memory limits, no access to hidden tests or server secrets, and shall be killed at the time limit. Verified by an isolation test suite.

NFR-07 Application security. OWASP ASVS level 1 basics: Argon2id password hashing, server-side role checks, input validation, and secrets kept out of the repository. Verified by automated tests and a ZAP baseline scan.

NFR-08 Availability. 99% availability during scheduled contest windows; losing one worker shall not lose a submission. Verified by a chaos test in the nightly suite.

NFR-09 Recoverability. Daily MongoDB backups; recovery point at most 24 h and recovery time at most 2 h. Verified by a restore drill once per release.

NFR-10 Auditability. 100% of privileged actions shall produce an audit record, and the application's database user shall only be able to insert into the audit collection, never update or delete. Verified by a database-privileges test and a coverage check.

NFR-11 Usability. The client shall work on current Chrome, Firefox and Edge, and the leaderboard shall be readable on phones. A full WCAG 2.1 AA pass is optional. Verified by axe-core checks in Playwright.

NFR-12 Maintainability. Module boundaries shall be enforced by lint rules; domain modules keep at least 70% line coverage; CI finishes in under 10 minutes. Verified by CI gates.

NFR-13 Testability. Integration tests shall run against real MongoDB and Redis using Testcontainers; a nightly invariant suite checks every business rule. Verified by the GitHub Actions nightly job.

NFR-14 Scalability. Judge capacity shall grow by adding workers, with no code change. Verified by adding a worker in a test environment and measuring throughput.

NFR-15 Observability. A health endpoint and structured JSON logs shall exist for the server and the judge worker; dashboards are optional. Verified by review.

NFR-16 Fairness. All contest and quiz timing shall use the server clock; the client clock is never trusted. Verified by unit tests with skewed clocks.

NFR-17 Cost. The system shall run within the free tiers of Vercel and MongoDB for the length of the course. Verified by a monthly billing check.

Table 3 lists the lifecycle transitions of the main objects and the requirements that trigger them. Table 4 lists the boundary cases that the tests must cover, with the expected result for each.

Object	**Transition**	**Requirement**
Contest	Draft → Registration open → Running → Frozen → Ended → Published	FR-05, FR-23, FR-24
Seat	(new) → Reserved	FR-06, FR-11
	(new) → Waitlisted	FR-07
	Reserved → Modified	FR-08
	Reserved / Modified → Withdrawn	FR-09
	Waitlisted → Reserved (promotion)	FR-09
	Reserved / Modified → Competing (contest starts; no check-in needed)	FR-10, FR-15
Problem version	Draft → Checking → Published / Blocked	FR-12, FR-14
Submission	Queued → Claimed → Compiling → Running	FR-16, FR-17
	Claimed / Running → Queued (reclaimed, new token)	FR-17
	Running → Judged → Scored	FR-16, FR-21
	Judged → Invalidated; Scored → Queued (rejudge)	FR-18

Table 3: Lifecycle transitions and the requirements that trigger them

Case	Values	Expected	Requirement
Race for the last seat	Two requests, one seat	One Reserved, one Waitlisted	FR-06, FR-07, NFR-04
Duplicate reservation	Same user, second request	Existing seat returned	FR-11
Withdraw, waitlist non-empty	Head at position 1	Head promoted, reserved unchanged	FR-09
Withdraw, waitlist empty	No waitlisted users	Reserved decremented by 1	FR-09
Submit without check-in	Reserved seat, contest running	Submission accepted	FR-10, FR-15
Worker dies mid-run	Entry idle past timeout	Reclaimed; stale commit matches nothing	FR-17, NFR-05
Simultaneous first solves	Two ACs on one problem	Bonus awarded once	FR-21
Unsolved problem	3 wrong attempts, no AC	Penalty 0	FR-21
Late quiz answer	$t > T_q$	Rejected	FR-20, NFR-16
Practice during a contest	Contest running	Practice uses at most 20% of slots	FR-19
Hidden contest problem	Unpublished contest	403 from the archive	FR-19
Weak tests	A wrong solution passes all tests	Publish blocked	FR-14

Table 4: Boundary cases and the requirements that govern them

1.5 Initial Requirements Traceability Matrix

A requirements traceability matrix links each requirement to where it came from and to the work that delivers it, so no requirement is lost or left unjustified. Table 5 traces every functional and non-functional requirement to its user story, the use case it supports, its planned test and its Jira issue. Optional rows are marked with an asterisk (*). The status of each row is updated at the end of every sprint.

Table 5: Initial requirements traceability matrix

ID	Requirement	Story	Use case	Planned test	Jira
FR-01	Sign up and sign in with JWT and refresh tokens	US-01	UC-01	Vitest unit + API test	CC-17
FR-02	Server-side checks for four roles	US-02	UC-01	Vitest unit + API test	CC-18
FR-03*	Grant and revoke roles, audited	US-03	UC-18	Vitest unit + API test	CC-39
FR-04	Create and configure a contest	US-04	UC-14	Vitest unit + API test	CC-19
FR-05	Contest state machine and scheduler	US-05	UC-14	Vitest unit + API test	CC-23
FR-06	Reserve a seat, never above capacity	US-06	UC-03	Concurrent integration test	CC-20

ID	Requirement	Story	Use case	Planned test	Jira
FR-07	FIFO waitlist with position	US-07	UC-04	Concurrent integration test	CC-21
FR-08*	Modify registration, keep seat	US-09	UC-05	Vitest unit + API test	CC-43
FR-09	Two-branch withdrawal with promotion	US-08	UC-05	`withdraw.spec.ts`	CC-22
FR-10*	Optional check-in, never required to submit	US-26	UC-06	Vitest unit + API test	CC-49
FR-11	One active seat per user per contest	US-06	UC-03	Concurrent integration test	CC-20
FR-12	Author problems with immutable versions	US-10	UC-11	Vitest unit + API test	CC-24
FR-13	Upload tests, reference and limits	US-11	UC-12	Playwright E2E	CC-25
FR-14	Publish check decided by sandbox verdicts	US-12	UC-13	`publishcheck.spec.ts`	CC-35
FR-15	Submit code with a reserved seat	US-13	UC-06, UC-07	Playwright E2E	CC-26
FR-16	Isolated judging, one verdict	US-13	UC-07	Judge test suite	CC-16, CC-26
FR-17	Exactly-once judging	US-14	UC-07	Worker-kill integration test	CC-27
FR-18*	Rejudge and invalidate	US-16	UC-16	Vitest unit + API test	CC-32
FR-19	Practice archive, slot share, 403 rule	US-24	UC-02	`practice.spec.ts`	CC-44
FR-20	Server-timed quiz, speed scoring	US-17	UC-08	`scoring.quiz.spec.ts`	CC-31
FR-21	ICPC standings and first-solve bonus	US-18	UC-09	`scoring.icpc.spec.ts`, `scoring.firstsolve.spec.ts`	CC-51, CC-30
FR-22	Push standings in real time	US-18	UC-09	Playwright E2E	CC-29
FR-23*	Frozen standings after freeze time	US-19	UC-17	Vitest unit + API test	CC-33
FR-24*	Publish final standings	US-19	UC-17	Vitest unit + API test	CC-33
FR-25*	Rating update after publish	US-25	UC-17	Vitest unit + API test	CC-45
FR-26*	Ask and answer clarifications	US-20	UC-10, UC-15	Vitest unit + API test	CC-36
FR-27*	Register, drain, evict judge workers	US-22	UC-19	Vitest unit + API test	CC-38
FR-28	Append-only audit collection	US-03, US-23	UC-20	Vitest unit + API test	CC-34
NFR-01	Standings reach clients in 1 s p95	US-18	UC-09	Load test + timing probe	CC-29
NFR-02	Judge turnaround at most 10 s p95	–	UC-07	Queue metrics	CC-27

ID	Requirement	Story	Use case	Planned test	Jira
NFR-03	API p95 at most 300 ms	–	All	Load test	CC-15
NFR-04	Reserved never above capacity	US-06	UC-03	Concurrent test + invariant query	CC-20, CC-41
NFR-05	One verdict per submission	US-14	UC-07	Worker-kill test + invariant query	CC-27, CC-41
NFR-06	Isolated execution with limits	–	UC-07	Isolation test suite	CC-16, CC-42
NFR-07	OWASP ASVS level 1 basics	–	All	Tests + ZAP baseline scan	CC-15
NFR-08	99% availability in contest windows	–	All	Chaos test (nightly)	CC-15
NFR-09	Daily backups, RPO 24 h, RTO 2 h	–	All	Restore drill	CC-46
NFR-10	Full audit coverage, records immutable	US-23	UC-20	DB privileges test	CC-34
NFR-11	Browser support, readable on phones	–	All	axe-core in Playwright	CC-47
NFR-12	Lint boundaries, 70% coverage, fast CI	–	All	CI gates	CC-3
NFR-13	Testcontainers, nightly invariant suite	–	All	GitHub Actions nightly job	CC-3, CC-41
NFR-14	Add judge workers with no code change	US-22	UC-19	Throughput measurement	CC-15
NFR-15	Health endpoint and JSON logs	–	All	Review	CC-48
NFR-16	Server clock only	US-05, US-17	UC-08, UC-14	Skewed-clock unit tests	CC-23, CC-31
NFR-17	Within the Vercel and MongoDB free tiers	–	All	Monthly billing check	CC-46

2 Architecture and Design

2.1 Component Diagram

Figure 2 shows the high-level architecture of CodeClash. CodeClash is a *modular monolith* built on the team's existing Express server, with a separate judge worker. All business modules live in one deployable, one folder per module. The judge worker runs outside it because running untrusted code has different risk and runtime needs from serving web requests. Everything is written in TypeScript, so the team uses one language and one test tool (Vitest). The application is hosted on Vercel, and MongoDB is the database.

- **Presentation layer (React 18 + Vite + Monaco):** separate areas for participants (`/arena`), problem setters (`/authoring`), organisers (`/control`) and platform admins (`/admin`).
- **Application layer (Express, Node, TypeScript):** an HTTP layer of REST routes and real-time channels, with middleware for authentication, roles, rate limits and audit, and the domain modules: Identity and Roles, Contest, Registration (seats and waitlist), Problem Library (versions and the publish check), Submission, Scoring and Leaderboard,

Clarification, and Admin and Audit. A shared layer holds the audit log, the transactional outbox dispatcher and the repositories.

- **Judge worker (TypeScript):** reads the judge queue through a Redis Streams consumer group, runs each submission in isolation with no network and with time and memory limits, and commits the verdict with a claim-token compare-and-set.
- **Data layer:** MongoDB is the source of truth for contests, seats, problems, submissions and verdicts. It enforces the concurrency rules with atomic single-document updates (`findOneAndUpdate` with a filter), unique partial indexes, multi-document transactions and JSON-schema validation. Redis holds the judge queue (Streams), the leaderboard read model (sorted sets) and rate limits.

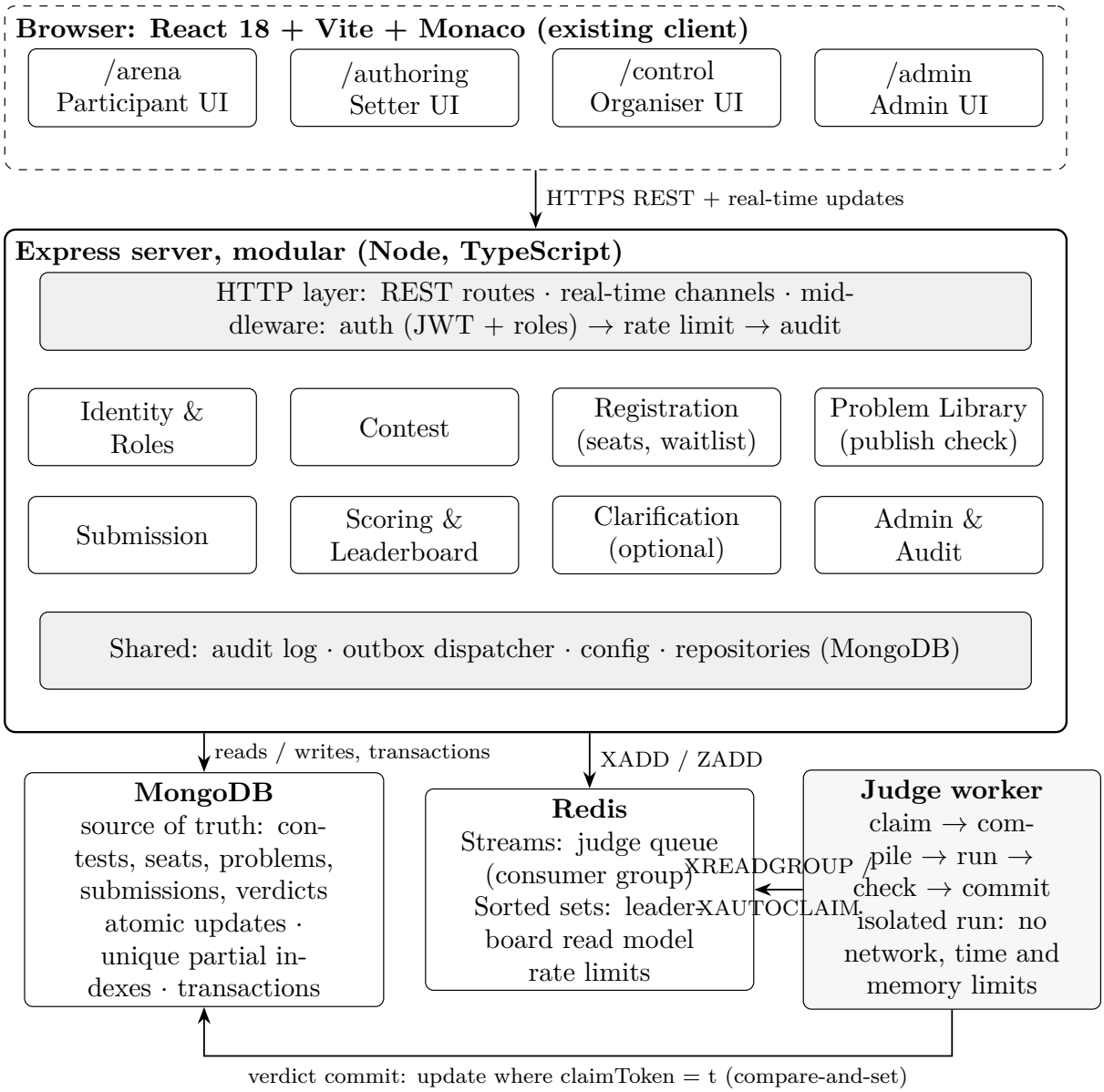


Figure 2: Component architecture of CodeClash: modular monolith, data stores and judge worker

2.2 Sequence Diagrams

To show the system's dynamic behaviour, we focus on the three workflows where concurrency and correctness matter most, followed by the lifecycles that guard them.

1. Seat Reservation, Waitlist and Withdrawal with Promotion: A seat is allocated by one atomic `findOneAndUpdate` on the contest document, whose filter only matches while `reserved < capacity`, and which increments `reserved` in the same operation. It runs in the same transaction as the seat insert. If a document is updated the seat is *Reserved*; if nothing matches the contest is full and a *Waitlisted* seat is inserted with the next position. A unique partial index on `(contestId, userId)` for active seats makes a repeated request return the existing seat. Withdrawal marks the seat *Withdrawn* and then atomically promotes the waitlist head (the waitlisted seat with the lowest position); if a head exists the reserved count stays the same, otherwise it is decremented. Both branches write an outbox event that pushes the change to clients. No application or Redis lock is used; the database decides. Figure 3 shows the messages and Figure 4 summarises the decision logic.

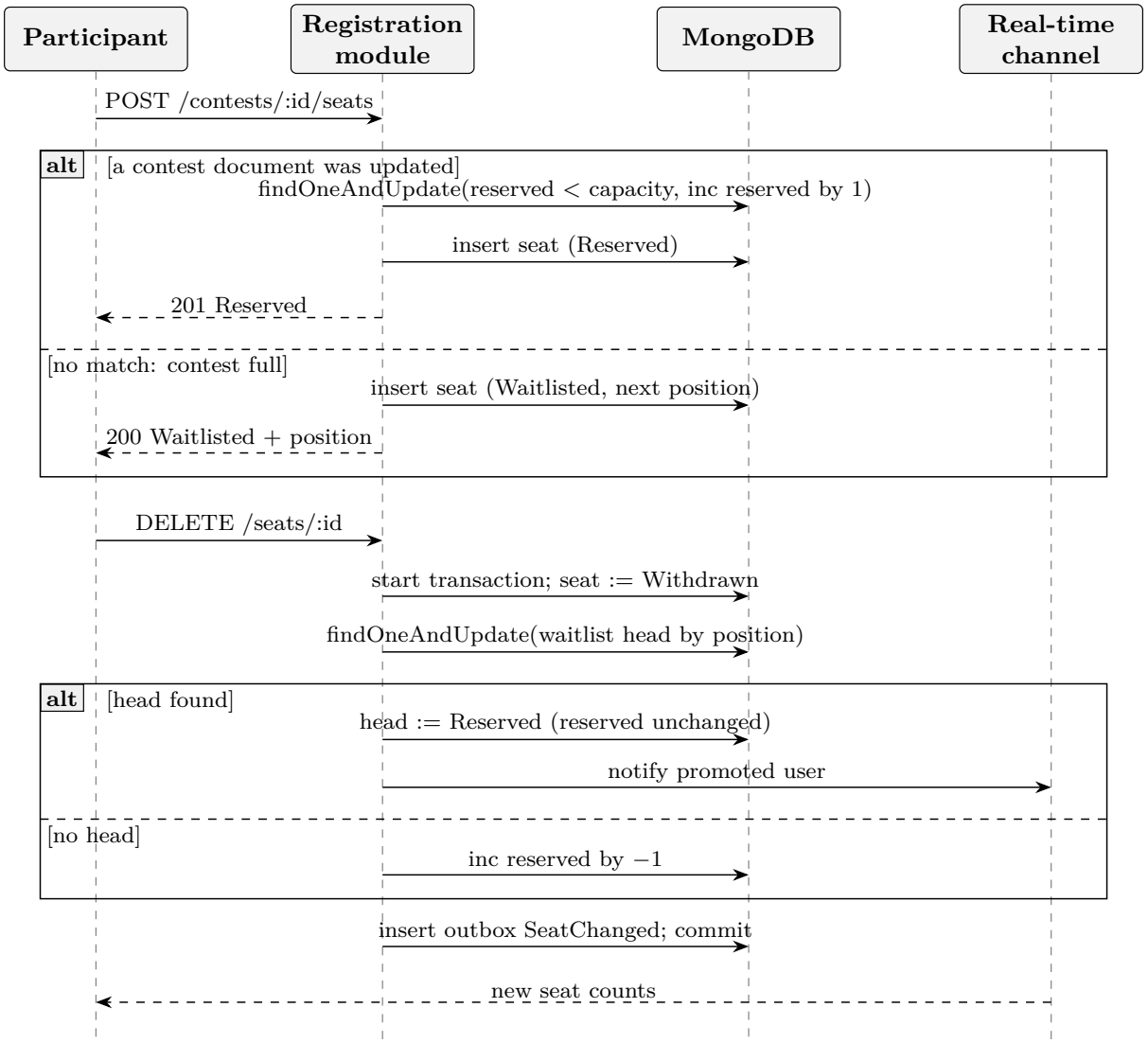


Figure 3: Sequence diagram for seat reservation, and withdrawal with both branches (promote head / decrement reserved)

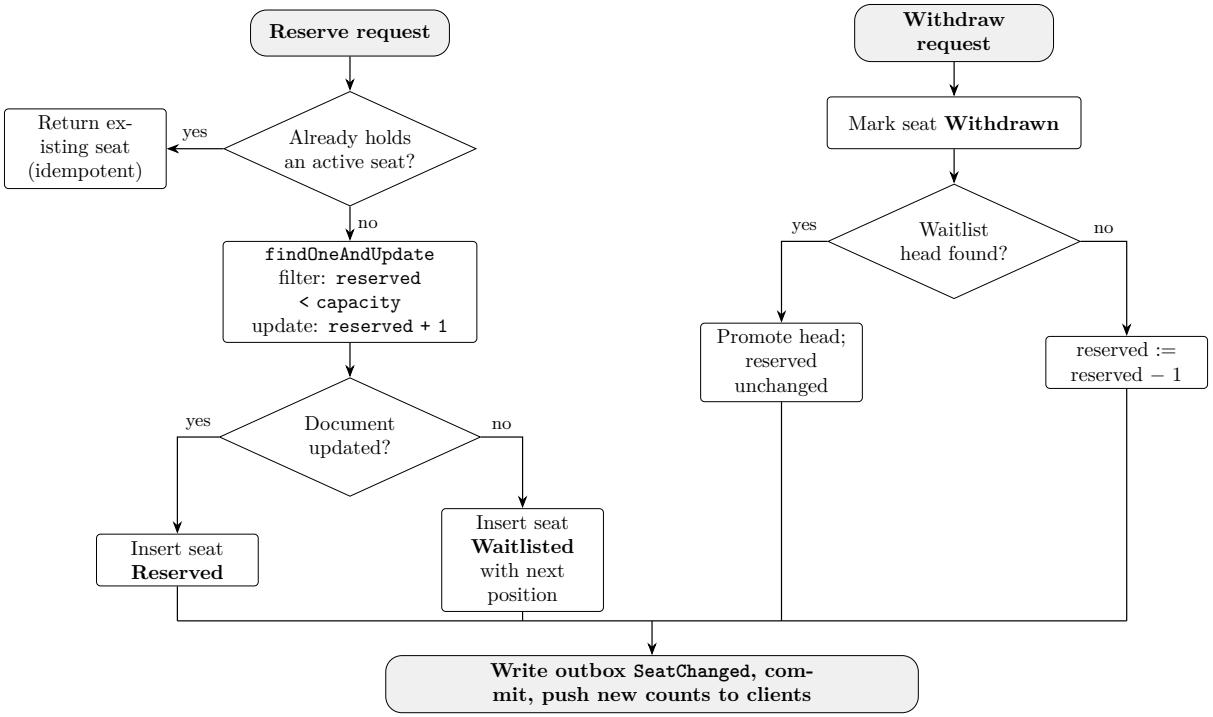


Figure 4: Decision flow for seat reservation and withdrawal; every path runs in one database transaction

2. Submission Judged Exactly Once: When a participant with a reserved seat submits code (check-in is not needed), the Submission module stores the submission as *Queued*, adds it to the judge stream and returns `202 Accepted` straight away. A judge worker reads the entry through the consumer group and writes a fresh random *claim token* to the submission document. It compiles once and runs the tests in isolation, stopping at the first failure. The verdict is saved with a compare-and-set: an `updateOne` whose filter matches both the submission ID and the claim token. If the worker dies, `XAUTOCLAIM` hands the idle entry to another worker with a new token, so the old worker's late commit matches no document and changes nothing. The outbox then applies scoring, updates the Redis leaderboard and pushes the change to every browser.

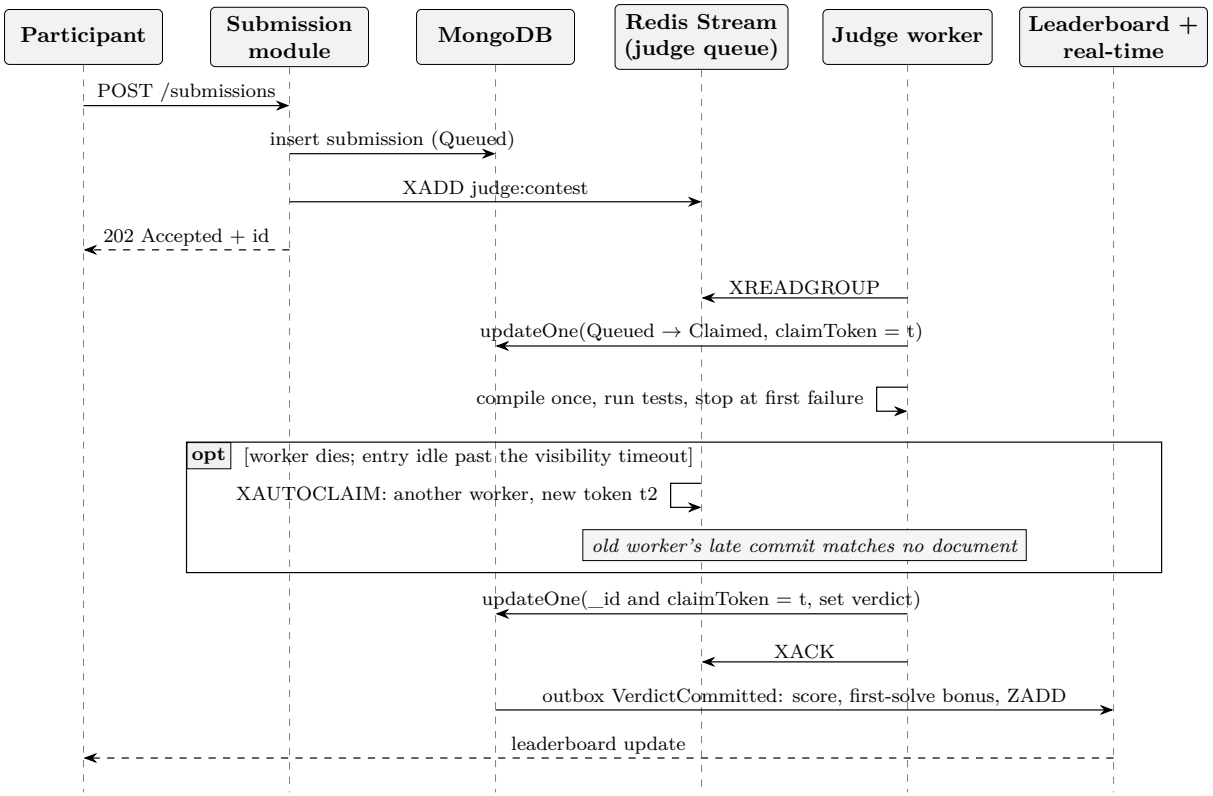


Figure 5: Sequence diagram for judging a submission exactly once, with consumer group, reclaim and claim-token compare-and-set

3. Publishing a Problem Through the Publish Check: When a setter asks to publish a problem version, the Problem Library sends the reference solution and every setter-supplied wrong solution to the judge. All of them run in the sandbox against every test. The version is published only if the reference gets AC on every test and each wrong solution fails at least one test; otherwise it is blocked and the report names the wrong solution that survived. The decision is taken only from the sandbox verdicts, so it is repeatable and can be tested.

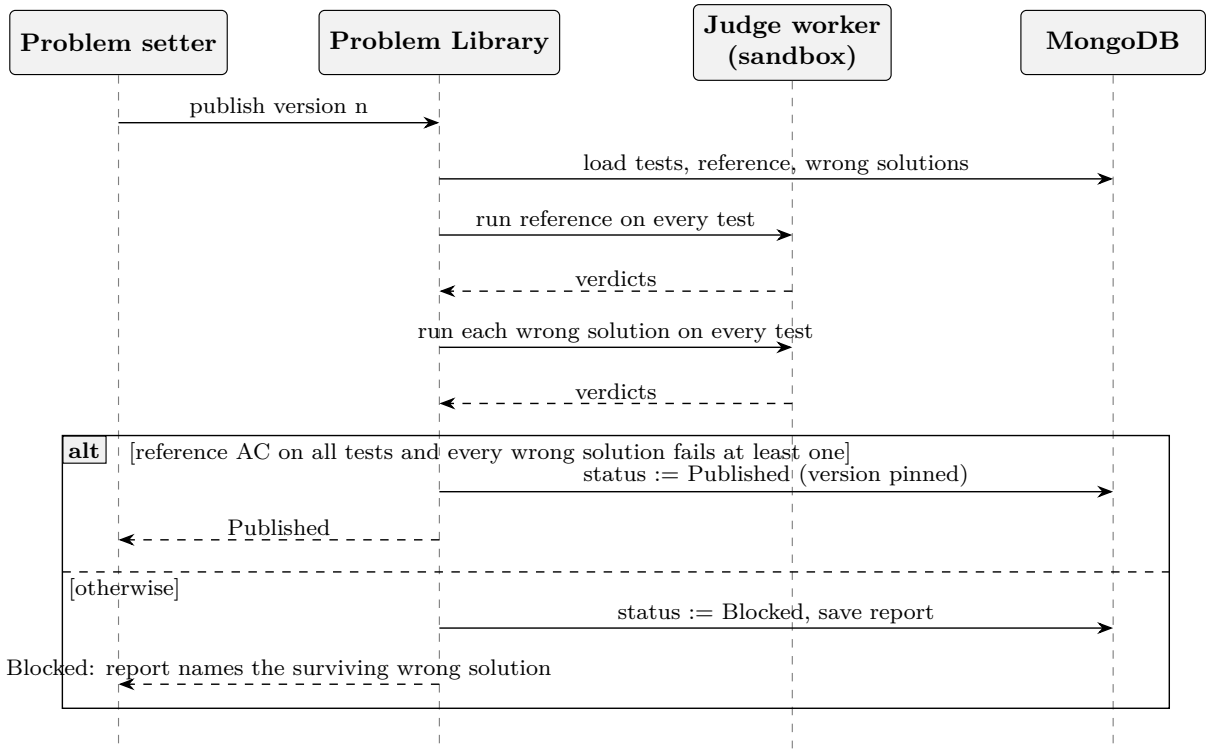


Figure 6: Sequence diagram for the publish check; only sandbox verdicts decide pass or fail

4. Seat and Submission Lifecycles: Figure 7 shows the states that a seat and a submission move through. Each transition is a guarded atomic update in MongoDB (the filter names the allowed current state), so an illegal move, such as a verdict on a submission that is not claimed, matches no document and is rejected by the database rather than by the user interface. Check-in is not a state: it only records attendance and never gates entering the contest or submitting.

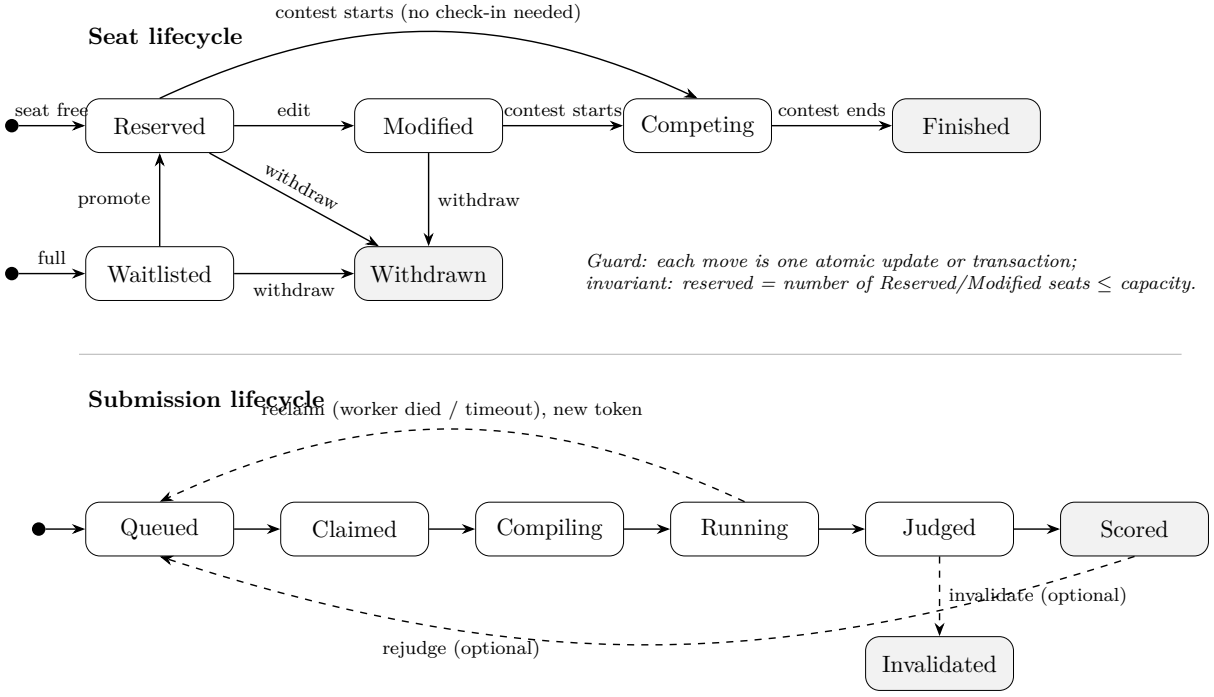


Figure 7: State machines for a seat and a submission, guarded by atomic updates in MongoDB

2.3 Design-Pattern Mapping

The architecture uses the following design patterns. Only patterns the team will actually build are listed; Factory, Command and Facade were in the first draft and were removed because nothing in the backlog implements them.

- **State Pattern:** applied to the seat, submission, problem-version and contest lifecycles (Table 3). Each state allows only its legal transitions; for example, a verdict can be committed only while the submission is Claimed, Compiling or Running. *(FR-05, FR-09, FR-14, FR-17)*
- **Optimistic Concurrency (compare-and-set):** the claim token on the verdict commit and the filtered `findOneAndUpdate` on the seat count. Stale workers and racing requests cannot overwrite each other. *(FR-06, FR-17)*
- **Competing Consumers:** judge workers share one Redis Streams consumer group, so each entry goes to one worker at a time and capacity grows by adding workers. *(FR-17, NFR-14)*
- **Idempotent Receiver:** seat reservation and verdict commit can be retried or redelivered without creating duplicates. *(FR-11, FR-17)*
- **Transactional Outbox:** seat changes and verdicts write an outbox document in the same MongoDB transaction, so events are never lost and never sent for rolled-back changes. *(FR-09, FR-22)*
- **Observer / Publish–Subscribe:** clients subscribe to a contest channel and receive standings changes as they happen. *(FR-22, NFR-01)*
- **CQRS (read model):** a Redis sorted set serves leaderboard reads while MongoDB handles writes. *(NFR-01)*
- **Strategy:** scoring modes (ICPC rule, quiz speed) and language runners can be swapped without changing their callers. *(FR-04, FR-15, FR-20)*
- **Template Method:** the judge pipeline (prepare → compile → run → check → report) has the same steps for every language, with overridable parts. The publish check reuses the same pipeline. *(FR-14, FR-16)*
- **Adapter:** the judge pipeline calls one runner interface to execute code, so the runner behind it can be replaced without changing the pipeline. *(NFR-06)*
- **Chain of Responsibility:** Express middleware runs cross-cutting checks in a fixed order: authentication → role → rate limit → audit. *(FR-02, FR-28)*
- **Layered modular monolith with Repository and Dependency Injection:** each module follows route → service → repository; services receive their repositories as constructor arguments, which keeps database queries out of domain logic and lets tests swap real and fake repositories. *(NFR-12, NFR-13)*

2.4 Architecture Alternatives and Rejected Decisions

We evaluated several approaches for the core engineering challenges. The main decisions are described below, each with the alternatives considered, the trade-offs and the final choice. Table 6 summarises all ten architecture decision records (ADRs).

Decision 1: Seat Allocation Under Concurrency (ADR-02)

- **Considered alternatives:** a Redis distributed lock (Redlock) around each reservation, or reading the seat count and then writing it back inside a transaction.
- **Trade-offs & rationale:** a Redis lock can expire in the middle of a reservation and is not tied to the database commit, so the lock and the data can disagree. Read-then-write needs retries on write conflicts and is easy to get wrong.
- **Final decision:** one atomic `findOneAndUpdate` whose filter requires `reserved < capacity`, plus a unique partial index on active seats. It is a single atomic operation, and the database itself guarantees that reserved seats never exceed capacity.

Decision 2: Exactly-Once Verdicts and the Judge Queue (ADR-03, ADR-04)

- **Considered alternatives:** RabbitMQ, Amazon SQS or BullMQ as the queue; trusting the queue's delivery guarantee, or a distributed lock per submission.
- **Trade-offs & rationale:** no queue can promise exactly-once delivery, and a lock can expire while a slow worker is still running. RabbitMQ is an extra service to run; SQS has no per-consumer pending list; BullMQ hides the claim details that we need to test.
- **Final decision:** a Redis Streams consumer group (exclusive claim, pending list and `XAUTOCLAIM`) plus a claim-token compare-and-set on the verdict commit, which makes a late commit harmless.

Decision 3: Primary Database (ADR-06)

- **Considered alternatives:** PostgreSQL (the earlier draft) and MySQL.
- **Trade-offs & rationale:** a relational database gives CHECK constraints and row locks, but CodeClash's main objects (problem versions with tests and limits, quiz rounds, submissions with verdict details) are nested documents that map naturally to MongoDB. Every invariant we need can be enforced in MongoDB: atomic filtered updates for counters and state changes, unique partial indexes for "one active seat" and "one first solve", and multi-document transactions for withdrawal with promotion.
- **Final decision:** MongoDB, with JSON-schema validation on each collection in place of CHECK constraints, and transactions enabled in every environment, including tests.

Decision 4: Overall Style and Framework (ADR-01, ADR-10)

- **Considered alternatives:** microservices, serverless-only functions, and a rewrite in NestJS and Next.js as proposed in Deliverable 0.
- **Trade-offs & rationale:** microservices add network calls, distributed transactions and operations work that a five-person team cannot carry. The rewrite would cost about two sprints before any new feature, while the repository already has rooms, real-time updates, a quiz mode and a Monaco editor.
- **Final decision:** keep and restructure the existing Express + React/Vite code as a modular monolith with a separate judge worker. Module folders and middleware give the same boundaries as a new framework.

Table 6: Summary of architecture decision records

ADR	Decision	Options considered	Chosen	Why the others were rejected
ADR-01	Overall style	Microservices; modular monolith; serverless-only	Modular monolith + judge worker	Microservices are too much operations work for the team.
ADR-02	Seat allocation	Redlock; read-then-write; atomic filtered update	Atomic `findOneAndUpdate` + unique partial index	Locks can expire; read-then-write needs retries.
ADR-03	Judge queue	RabbitMQ; SQS; BullMQ; Redis Streams	Redis Streams consumer group	Extra services, no pending list, or hidden claim details.
ADR-04	Exactly-once verdicts	Trust queue; lock per submission; claim token	Claim-token compare-and-set	Queues cannot promise exactly-once; locks expire.
ADR-05	Real-time transport	Polling; Server-Sent Events; WebSocket (Socket.IO)	Socket.IO (already in the code)	Polling misses 1 s cheaply; SSE is one-way.
ADR-06	Primary database	PostgreSQL; MySQL; MongoDB	MongoDB	Document model fits our data; atomic updates, unique indexes and transactions cover every invariant.
ADR-07	Leaderboard reads	Aggregation per request; scheduled materialised collection; Redis sorted set	Redis sorted set	Too slow at 200 clients, or refreshed too rarely.
ADR-08	Problem quality gate	Manual review; AI-generated tests; sandbox verdicts on reference and wrong solutions	Sandbox verdicts only	Manual review misses weak tests; a model's report is not repeatable and must not decide.
ADR-09	Hosting	AWS EC2 + RDS (earlier draft); Kubernetes; Vercel	Vercel	EC2 needs manual server upkeep; Kubernetes costs money and effort; Vercel gives free-tier hosting with a preview for every pull request.
ADR-10	Framework	NestJS + Next.js rewrite; keep Express + React/Vite	Keep and restructure existing code	The rewrite costs about two sprints.

Evaluated and Rejected LLM Recommendations

During architecture planning we used an LLM to draft the architecture, pattern mapping and decisions. We reviewed its output and rejected the following recommendations:

1. **Recommendation: add Factory, Command and Facade to the pattern mapping.**
   Rejection rationale: nothing in the backlog implements them. Listing patterns that will not be built would make the mapping untrue, so only patterns with a matching Jira issue

were kept.

2. **Recommendation: keep the Deliverable 0 stack (NestJS + Next.js).**
   Rejection rationale: the existing repository already provides rooms, real-time updates, a quiz mode and a Monaco editor on Express + React/Vite. A rewrite would cost about two sprints of a four-sprint plan (ADR-10).

3. **Recommendation: capacity, throughput and velocity figures produced by the LLM.**
   Rejection rationale: arithmetic on queues, capacity and team speed was the LLM's most common error. No figure was accepted as generated; every target in this document (for example, about 86 mandatory points for four sprints) was recomputed by the team and shows its calculation.

3 Project Management Setup

3.1 Jira Project and Initial Backlog

The project is managed in Jira under the project key **CC** on a team-owned site:

`https://cs455-codeclash.atlassian.net/jira/software/projects/CC/boards/1/backlog`

The project uses the Scrum template (team-managed) with the issue types Epic, Story, Task, Bug, Feature and Sub-task. The backlog is organised into ten epics, one for each major functional area of CodeClash. CC-1 to CC-3 are Sprint 1 setup tasks, CC-4 is a sub-task of CC-2, and the remaining issues are stories and tasks; CC-51 is the scoring-engine story added after review. Every issue carries its epic, owner, story points, sprint label and the requirement IDs it implements in its description. The ten optional items carry the label `optional` and have no sprint. The project members are the five students (Administrators), and Prof. Priyanka Bagade and TA Ankit Kumar (Viewers).

Epic	Jira area	Requirements	Jira issues
CC-5	E1 Project setup, CI/CD and deployment	NFR-12, NFR-13, NFR-17	CC-1 – CC-4, CC-46
CC-6	E2 Identity and roles	FR-01, FR-02	CC-17, CC-18
CC-7	E3 Contest management	FR-04, FR-05, FR-26, NFR-16	CC-19, CC-23, CC-36
CC-8	E4 Registration, seats and waitlist	FR-06 – FR-11	CC-20 – CC-22, CC-43, CC-49
CC-9	E5 Problem library and publish check	FR-12 – FR-14	CC-24, CC-25, CC-35
CC-10	E6 Submission and judge pipeline	FR-15 – FR-19, NFR-06	CC-16, CC-26, CC-27, CC-32, CC-44
CC-11	E7 Scoring, leaderboard and rating	FR-21 – FR-25, NFR-01	CC-29, CC-30, CC-33, CC-45, CC-51
CC-12	E8 Quiz engine	FR-20	CC-31
CC-14	E10 Admin, fleet and audit	FR-03, FR-27, FR-28	CC-34, CC-38, CC-39
CC-15	E11 Non-functional and invariant testing	NFR-02 – NFR-05, NFR-11, NFR-15	CC-41, CC-42, CC-47, CC-48

Table 7: Organisation of the initial Jira backlog

All issues start in the *To Do* state because implementation has not yet begun. The workflow is To Do → In Progress → In Review → Done, with *Blocked* used as a flag. An issue moves to In Review when its pull request is ready for peer review, and it is marked Done only when it meets the Definition of Done: code reviewed and merged to `main`, CI green, tests for its acceptance criteria, RTM row updated, and deployed to a Vercel preview. An issue is *ready* for a sprint when it has a requirement ID, acceptance criteria, an estimate and no open blocker.

Table 8 lists the full backlog (the same list is exported as `CodeClash_Jira_Backlog.csv`).

Table 8: Initial Jira backlog

Key	Type	Summary	Epic	Owner	Pts	Sprint
CC-1	Feature	Local dev stack: MongoDB (with transactions), Redis, api, web, worker	CC-5	Srujan	2	1
CC-2	Task	Restructure the existing repo into modules; add MongoDB and Redis	CC-5	Srujan	2	1
CC-3	Feature	GitHub Actions pipeline: lint, unit, integration (Testcontainers), build	CC-5	Srujan	3	1
CC-4	Sub-task	Create module folders and update README (sub-task of CC-2)	CC-5	Srujan	–	1
CC-16	Task	Isolated code execution: no network, time and memory limits, kill at time limit	CC-10	Om	5	1
CC-17	Story	US-01 Sign up and sign in with JWT + refresh tokens	CC-6	Srujan	3	1
CC-26	Story	US-13 Submit code and receive verdict	CC-10	Om	5	1
CC-27	Story	US-14 Exactly-once judging: consumer group, XAUTOCLAIM, claim-token CAS	CC-10	Om	8	1
CC-34	Story	Append-only audit log for privileged actions	CC-14	Ridham	3	1
CC-18	Story	US-02 Server-side role checks for four roles	CC-6	Srujan	2	2
CC-19	Story	US-04 Create and configure contest	CC-7	Aritra	3	2
CC-20	Story	US-06 Reserve seat with atomic findOneAndUpdate + unique partial index	CC-8	Aritra	8	2
CC-21	Story	US-07 Waitlist with FIFO positions	CC-8	Aritra	2	2
CC-22	Story	US-08 Withdraw: promote waitlist head or decrement reserved	CC-8	Aritra	3	2
CC-23	Story	US-05 Contest state machine and scheduler	CC-7	Aritra	2	2
CC-29	Story	US-18 Live leaderboard: real-time push + Redis sorted set	CC-11	Peeyush	5	2
CC-51	Story	Scoring engine: ICPC penalty, quiz speed and first-solve formulas	CC-11	Peeyush	5	2
CC-24	Story	US-10 Problem authoring with versions	CC-9	Ridham	3	3
CC-25	Story	US-11 Upload tests, reference, wrong solutions and limits; create demo problems	CC-9	Ridham	3	3
CC-30	Story	First-solve bonus awarded exactly once	CC-11	Peeyush	2	3
CC-41	Task	Nightly invariant suite (MongoDB aggregation checks)	CC-15	Peeyush	3	3
CC-42	Task	Execution isolation test suite	CC-15	Ridham	3	3
CC-46	Task	Deploy to Vercel (preview per pull request and production)	CC-5	Srujan	3	3
CC-31	Story	US-17 Server-timed quiz rounds scored on speed	CC-12	Srujan	2	4
CC-35	Story	US-12 Publish check: sandbox verdicts on reference and wrong solutions decide	CC-9	Ridham	3	4

Key	Type	Summary	Epic	Owner	Pts	Sprint
CC-44	Story	US-24 Practice archive: contest slot share; unpublished problems blocked	CC-10	Peeyush	3	4
CC-32	Story	US-16 Rejudge and invalidate	CC-10	–	5	Opt.
CC-33	Story	US-19 Freeze and publish standings	CC-11	–	3	Opt.
CC-36	Story	US-20 Clarifications (organisers answer privately or broadcast)	CC-7	–	5	Opt.
CC-38	Story	US-22 Register, drain and evict judge workers	CC-14	–	5	Opt.
CC-39	Story	US-03 / US-23 Role management and audit search	CC-14	–	3	Opt.
CC-43	Story	US-09 Modify registration	CC-8	–	2	Opt.
CC-45	Story	US-25 Rating update after contest	CC-11	–	3	Opt.
CC-47	Task	Accessibility pass (axe-core) and cross-browser check	CC-15	–	3	Opt.
CC-48	Task	Observability: structured logs, metrics, health endpoints	CC-15	–	3	Opt.
CC-49	Story	US-26 Optional check-in (attendance only, never required to submit)	CC-8	–	3	Opt.

Figure 8 shows the initial product backlog in Jira.

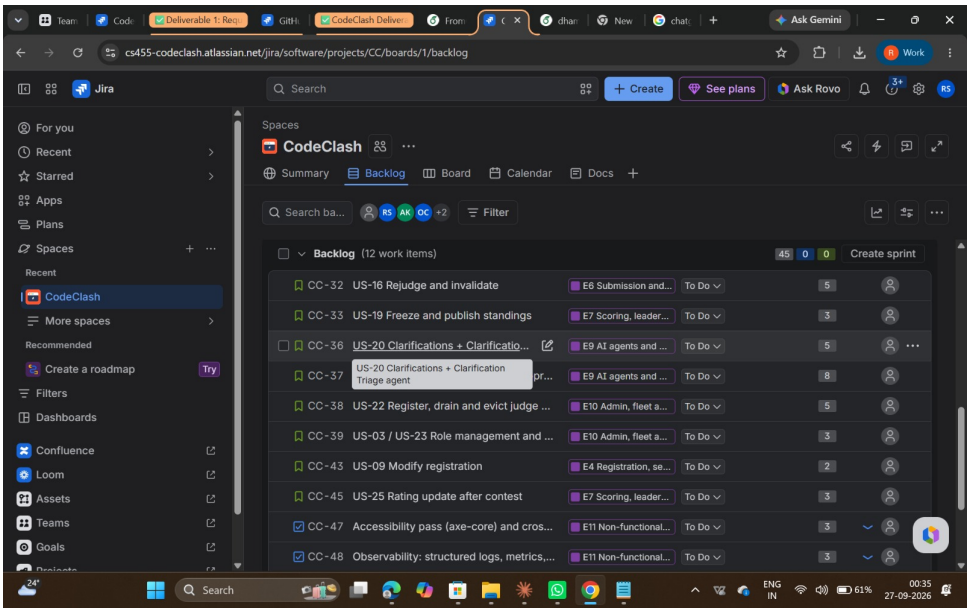


Figure 8: Initial Jira product backlog for CodeClash

3.2 Jira–GitHub Integration

The Jira site has the *GitHub for Atlassian* app installed. The repository to be connected is:

`https://github.com/SrujanBhirud/CodeClash`

Because the repository is owned by Srujan's personal GitHub account, only he can authorise the Jira app for it. He has been made a Jira administrator for this purpose and connects the repository, selecting only `SrujanBhirud/CodeClash`. Once the backfill finishes, the Development panel of each issue shows its branches, commits and pull requests. When Jira was checked on 26 September 2026, no CC issue had repository data yet; evidence of the Development panel will be added as soon as the connection is made (Figure 9).

The team uses Jira issue keys consistently in branch names, commit messages and pull request titles, as shown in Table 9. For example, the work for CC-20 uses the branch `feature/CC-20-seat-reservation`.

Item	Convention	Example
Branch	`<type>/CC-<key>-<short-name>`	`feature/CC-20-seat-reservation`
Commit	`CC-<key> <imperative summary>`; smart commits allowed	`CC-20 add atomic seat update #comment covered by concurrency test`
Pull request title	`CC-<key>: <summary>`	`CC-20: Reserve seat with atomic update`
Pull request body	Links the issue, lists tests, notes the RTM row	Closes CC-20; Tests: `seat.concurrency.spec.ts`; RTM: FR-06

Table 9: Branch, commit and pull request conventions

The `main` branch is protected: every change needs a pull request, one approving review and green CI checks, branches must be up to date, and force pushes are blocked. A pull-request template asks for the Jira key, what changed, how it was tested and which RTM row was updated. Vercel builds a preview deployment for every pull request, so reviewers can try the change. Jira automation moves an issue to In Progress when a branch is created, to In Review when a pull request is opened, and to Done when it is merged. The required CI checks are:

- **lint-and-typecheck:** ESLint (with module-boundary rules) and `tsc`;
- **unit:** Vitest for the client, server and judge worker;
- **integration:** server tests against MongoDB and Redis in Testcontainers;
- **build:** production build of the client, server and judge worker;
- **nightly (not blocking):** the invariant suite and the execution isolation tests.

Check	Status / owner
GitHub for Atlassian app installed on cs455-codeclash	Done, 26 Sep (Ridham)
Repository owner given Jira admin to authorise the connection	Done, 26 Sep (Srujan)
`SrujanBhirud/CodeClash` connected, only this repository selected	Srujan, on accepting the Jira invite
Branch `feature/CC-1-test` and a `CC-1` commit appear in CC-1's Development panel	Checked right after the connection
Pull request `CC-1: ...` shows with its CI status	First pull request of Sprint 1
Branch protection on `main`	Srujan, Sprint 1 day 1
Instructor and TA in the Jira project	Done, 26 Sep (Viewers)
Instructor and TA on the GitHub repository	Srujan, Sprint 1 day 1

Table 10: Integration verification status

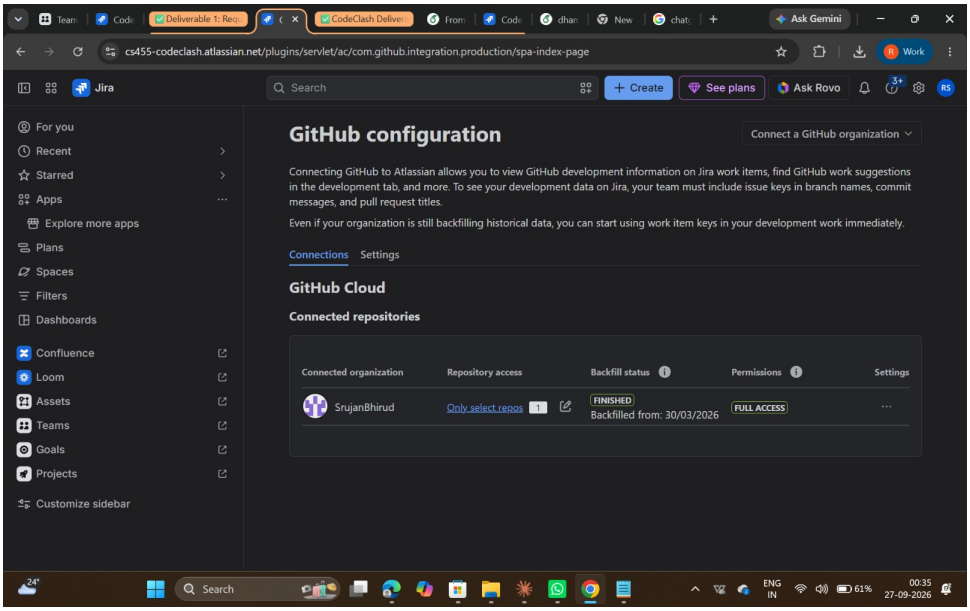


Figure 9: Jira–GitHub integration and the connected CodeClash repository

3.3 Initial Sprint Plan

Development is organised into four sprints ending on 14 November 2026, after a setup sprint (Sprint 0) that produced this deliverable. Two internal milestones fall on sprint ends: 15 October (end of Sprint 1) and 6 November (end of Sprint 3). The course demo takes place between 8 and 13 November, inside Sprint 4. Planning happens on the first day of each sprint, stand-ups three times a week, and the review and retrospective on the last day.

Working software for the demo. Each increment has to be working software, and the demo has to use realistic content. Problems must therefore exist before the demo, not be written during it. Problem authoring and test upload (CC-24, CC-25) are placed in Sprint 3, which ends on 5 November, and the demo problems are created there. The early demo uses those problems together with the judge built in Sprint 1. Sprint 4 is kept light because it overlaps the demo: it finishes the publish check, quiz rounds and the practice rules.

Capacity. An ideal pace for five students is about 17–18 points a week, but first-time teams usually reach 60–70% of their plan. The plan therefore uses about 14 points a week, so roughly 100 points would fit between 28 September and 14 November. The mandatory scope is 86 points, which leaves a buffer for the demo week. The first draft had 170 points and a fifth sprint ending 4 December; to fit mid-November, ten items (35 points) were moved to optional, the existing Express + React/Vite code is kept instead of a rewrite, and the AI agents, the per-user submission quota and the burst tests were removed from scope.

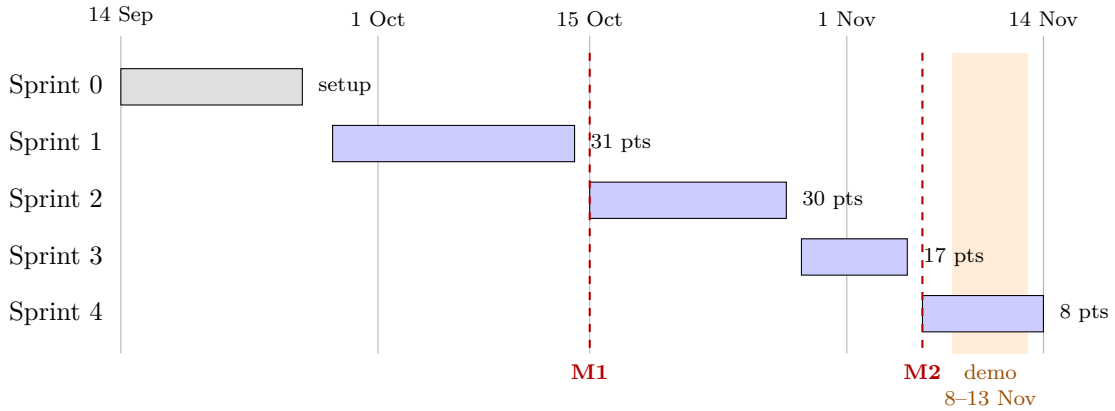


Figure 10: Sprint timeline. M1 (15 Oct): sign-in, a judged submission and the exactly-once commit. M2 (6 Nov): authoring and test upload with demo problems, Vercel deployment, invariant and isolation tests. The shaded band is the demo window.

Sprint	Dates	Goal	Issues	Pts
0	14 – 26 Sep 2026	Requirements, architecture, Jira and GitHub setup (this deliverable); closed in Jira	–	–
1	28 Sep – 14 Oct 2026	Milestone 15 Oct: sign-in, a submission judged in isolation, the claim-token exactly-once commit, and the audit log	CC-1 – 4, 16, 17, 26, 27, 34	31
2	15 – 28 Oct 2026	Roles, contest setup and state machine, seat reservation, waitlist and two-branch withdrawal, live leaderboard, scoring engine	CC-18 – 23, 29, 51	30
3	29 Oct – 5 Nov 2026	Milestone 6 Nov: problem authoring and test upload with the demo problems, first-solve bonus, invariant suite, isolation tests, Vercel deployment	CC-24, 25, 30, 41, 42, 46	17
4	6 – 14 Nov 2026	Demo 8–13 Nov. Publish check, server-timed quiz rounds, practice archive rules	CC-31, 35, 44	8
Opt.	After 14 Nov or if early	Rejudge, freeze/publish, clarifications, fleet admin, audit search, modify registration, rating, accessibility, observability, check-in	10 items	35

Table 11: Planned sprint roadmap

Sprint 1 goal. The first sprint delivers milestone M1: a signed-in participant submits code that is judged in isolation, the verdict is committed through the claim-token compare-and-set (a killed worker never writes), and privileged actions are recorded in the append-only audit log. Seat reservation follows in Sprint 2. The sprint contains the nine work items in Table 12, all in the To Do state at the start of the sprint.

Issue	Work	Owner	Pts
CC-1	Local dev stack: MongoDB (with transactions), Redis, api, web, worker	Srujan	2
CC-2	Restructure the existing repo (Express server, React + Vite client) into modules; add MongoDB and Redis	Srujan	2
CC-3	GitHub Actions pipeline: lint, unit, integration (Testcontainers), build	Srujan	3
CC-4	Create module folders and update README (sub-task of CC-2)	Srujan	–
CC-16	Isolated code execution: no network, time and memory limits, kill at time limit	Om	5
CC-17	US-01 Sign up and sign in with JWT + refresh tokens	Srujan	3
CC-26	US-13 Submit code and receive verdict	Om	5
CC-27	US-14 Exactly-once judging: consumer group, XAUTOCLAIM, claim-token CAS	Om	8
CC-34	Append-only audit log for privileged actions	Ridham	3
	Total		**31**

Table 12: Initial Sprint 1 backlog

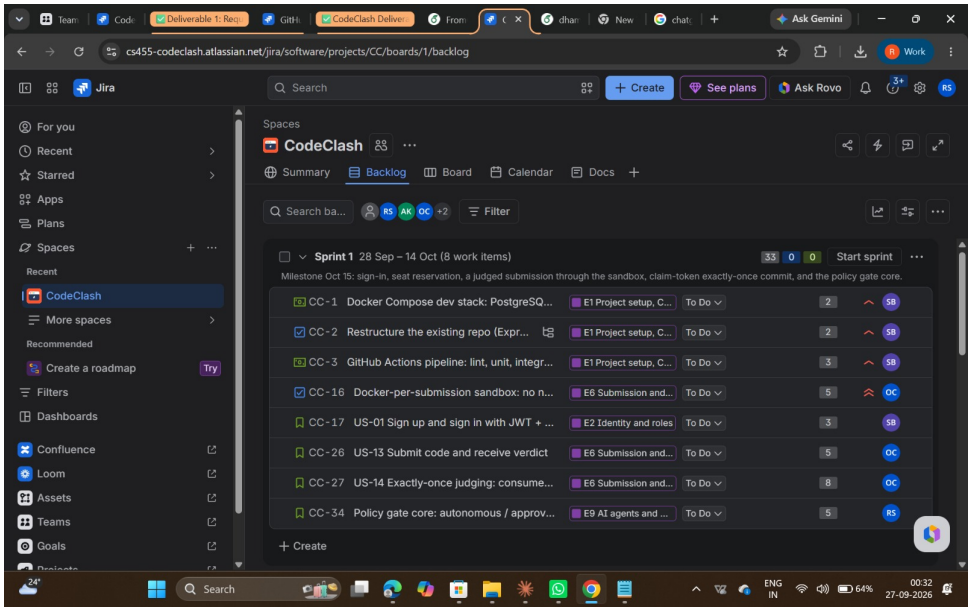


Figure 11: Initial Sprint 1 backlog in Jira

Ownership. Each student owns roughly a fifth of the mandatory work (15–18 points each). Owners are the Jira assignees and can swap tasks at sprint planning.

Owner	Owns	Pts
Srujan Sachin Bhirud	E1 setup, CI and Vercel deployment (CC-1, 2, 3, 46), E2 sign-in and roles (CC-17, 18), quiz rounds (CC-31)	17
Aritra Kolay	E3 contest setup and state machine (CC-19, 23), E4 seats, waitlist and withdrawal (CC-20, 21, 22)	18
Om Chandrakant Chaudhari	Judge pipeline: isolated execution, submit and verdict, exactly-once judging (CC-16, 26, 27)	18
Peeyush Agarwal	E7 leaderboard and scoring (CC-29, 30, 51), invariant suite (CC-41), practice rules (CC-44)	18
Ridham Divyakant Solanki	Audit log (CC-34), E5 problem authoring, test upload and publish check (CC-24, 25, 35), isolation tests (CC-42)	15

Table 13: Ownership of the mandatory work

Risks for the first sprints.

- **Running untrusted code (high):** no network, time and memory limits and a time kill from day one (CC-16); isolation test suite (CC-42) by 6 November.
- **No realistic content for the demo (high):** authoring and test upload moved to Sprint 3, so the demo problems exist before 8 November.
- **Concurrency bugs that appear only under concurrent access (high):** integration tests against real MongoDB from Sprint 1, and the nightly invariant suite from Sprint 3.
- **MongoDB transactions need a replica set (medium):** the local stack, CI and deployment all run MongoDB with transactions enabled, and the withdrawal tests fail if transactions are unavailable.
- **Sprint 1 is the heaviest at 31 points (medium):** CC-1 and CC-3 are small and finished in the first days; the M1 items are protected.
- **Scope still too large for 14 November (medium):** the optional list already holds 35 points; if Sprint 1 velocity is below 25 points, CC-31 and CC-44 move to optional next.
- **Judge throughput misses its target at peak (medium):** compile once and exit early on the first failure.

An issue is complete only when it satisfies its requirement and acceptance criteria, its GitHub changes have been reviewed by another team member, the pull request has been merged, and the Jira issue has been updated. Incomplete issues stay open and are carried forward rather than being marked Done artificially.

4 AI Engineering Log for Requirements and Architecture Activities

LLMs were used during the requirements, architecture and project-setup phases of CodeClash. The tools were an AI chatbot (for the GitHub auto-PR workflow) and Claude (Anthropic) through Claude Code (documents and diagrams) and Claude in Chrome (Jira and GitHub in the browser). The outputs were treated as drafts, not as authoritative answers. Prompts below are shown in a cleaned-up, paraphrased form with the same meaning as what we asked; the exact chats are kept in each member's chat history. The team followed four rules:

- AI output is a draft; a named team member checks it before it enters Jira, the RTM or the repository.
- No passwords, tokens or personal data are typed into AI tools; logging in and approving access are done by a person.
- Every number the AI produces (targets, estimates, capacities) is checked by arithmetic or against Jira.
- Any AI action that changes a shared system (Jira, GitHub) is approved in chat first, and deletions are never delegated.

4.1 AI-Assisted Requirements Refinement and Planning

4.1.1 Objective

To turn the requirements into a use-case model, find risks we had not seen, and produce a sprint plan and Jira backlog that fit the real mid-November deadline. The aim was that every requirement be:

- covered by a use case and a user story;
- testable, with a measurable target;
- planned in a sprint or explicitly marked optional;
- present as a Jira issue.

4.1.2 Prompt / Interaction

The context of CodeClash (the Deliverable 0 proposal and our requirements) was given first, followed by these prompts:

> *Draw the use-case diagram for all user roles.*
> *List any additional risks that we may not have seen.*
> *Make a sprint plan that finishes by mid-November.*
> *Create the backlog and sprints in Jira to match the plan.*

4.1.3 LLM Output

- A use-case diagram with the human actors, the system actors and their use cases.
- Additional risks, for example concurrency bugs that appear only under concurrent access.
- A plan of four sprints ending 14 November, with optional items and equal owners.
- In Jira: epics, the stories and tasks with points and priorities, and Sprints 1–4.

4.1.4 Team Evaluation

The team checked that every use case in the diagram appears in the use-case catalogue, and that each new risk had a concrete response. The sprint dates were checked against the real deadline and against the demo window (8–13 November), and the keys, points and sprints in Jira were checked one by one against this document. We found that a plan must be checked against the real deadline and the demo, not only against a feature list: the first draft (170 points and a fifth sprint ending 4 December) did not fit, and a later version put problem authoring in the demo week.

4.1.5 Changes Made

- Two internal milestones were set: 15 October and 6 November.
- Optional items were labelled in Jira, and the AI agents, the per-user submission quota and the burst tests were removed from scope.
- Problem authoring and test upload were moved to Sprint 3 so that demo problems exist before the demo.
- Submitting was made independent of check-in; check-in is optional and only records attendance.
- Every capacity and velocity figure was recomputed and now shows its calculation.
- Writing out the ICPC penalty rule as a formula exposed a missing test (an unsolved problem must add zero penalty), which became part of CC-51.

4.1.6 Final Decision

The LLM output was **modified and partially accepted**. The use-case model and risk list were accepted after checking; the sprint plan was accepted only after it was re-cut to the real deadline and the demo window, and its numbers were recomputed.

4.2 AI-Assisted Architecture Generation

4.2.1 Objective

To draft the architecture, the component, sequence and state diagrams, the design-pattern mapping and the decision records with the options we rejected, and to check them against our requirements and our existing code.

4.2.2 Prompt / Interaction

The context of CodeClash was given first, followed by these prompts:

> *Write the architecture, design patterns and design decisions with the options we rejected.*
> *Draw the component diagram and the sequence diagrams for seat reservation (with waitlist and withdrawal) and judging a submission exactly once.*
> *Draw the state diagrams for a seat and a submission.*

4.2.3 LLM Output

- An architecture description, a design-pattern mapping and a set of ADRs.
- A component diagram and sequence diagrams.
- State machines for a seat and a submission.

4.2.4 Team Evaluation

The team evaluated the output on:

- whether each pattern is really implemented by a backlog item;

- fit with the existing repository (Express, React/Vite, real-time updates, quiz mode);
- concurrency correctness of the seat and judging designs;
- risk for a mid-November finish;
- consistency of the diagrams with the module list and the business rules.

4.2.5 Changes Made

- Factory, Command and Facade were removed from the pattern mapping.
- The stack was changed from a NestJS + Next.js rewrite to our existing Express + React/Vite code (ADR-10).
- The team chose MongoDB as the database and Vercel for hosting, and the concurrency design was rewritten with atomic filtered updates, unique partial indexes and transactions.
- The problem quality gate was made deterministic: the publish check is decided only by sandbox verdicts, not by a model's report (ADR-08).
- The diagrams were redrawn to match the module list, and every state and transition in the state machines was checked against the business rules.

4.2.6 Final Architecture

The selected architecture is a **modular monolith** on the existing Express server, with a separate judge worker. Its main parts are:

1. **React 18 + Vite + Monaco client:** separate areas for participants, setters, organisers and admins.
2. **Express server:** one deployable with a folder per domain module, and middleware for authentication, roles, rate limits and audit.
3. **MongoDB:** the source of truth; atomic filtered updates, unique partial indexes and transactions enforce the seat, verdict and first-solve rules.
4. **Redis:** the judge queue (Streams consumer group), the leaderboard read model (sorted sets) and rate limits.
5. **Judge worker:** runs each submission in isolation, with the claim-token compare-and-set for exactly-once verdicts; the publish check reuses the same pipeline.
6. **Vercel:** hosting, with a preview deployment for every pull request.

The resulting high-level workflow for a submission is:

Participant → Express server → MongoDB + Redis Stream → Judge worker → claim-token commit → Outbox → Leaderboard → all browsers

and for publishing a problem:

Setter → Problem Library → Judge worker (reference and wrong solutions) → Verdicts → Published / Blocked

4.3 AI-Assisted Project Setup

4.3.1 Objective

To automate pull-request creation and connect Jira to the GitHub repository.

4.3.2 Prompt / Interaction

> *Write a GitHub Actions workflow that opens a pull request into main whenever another branch is pushed.*
>
> *Connect our Jira site to our GitHub repository.*

4.3.3 LLM Output

- `auto-pr.yaml`: runs on a push to any branch except `main` and opens a pull request with `gh pr create`.
- The GitHub for Atlassian app was installed on the Jira site, with written steps for the repository owner, who must authorise it.

4.3.4 Team Evaluation and Final Decision

The workflow was tested on the branch `autoPR_test` and then used for pull request #3; it was **accepted**, with the rule that branch names must carry the Jira key so that Jira links the pull request. For the integration, the AI's browser actions saved time in Jira, but anything touching a login or another team's work was stopped for a human decision: Srujan was made a Jira admin and completes the link from his own GitHub account (Section 3.2).

4.4 Lessons

- The AI's arithmetic on queues, capacity and team speed was its most common error; every target now shows its calculation.
- Words like "ICPC penalty" hide rules; writing the formula exposed a missing test.
- A plan has to be checked against the real deadline and the demo date, not only against a feature list.
- Browser actions by the AI saved time in Jira, but anything touching another team's work or a login was stopped for a human decision.
- Reusing working code (rooms, real-time updates, quiz, the auto-PR workflow) was worth more than a cleaner rewrite.